

The *Jason* Agent Programming Language

We usually find it convenient to make a distinction between an *agent program* and an *agent architecture*. The agent architecture is the software framework within which an agent program runs. The PRS is an example of an agent architecture; the plans are the program that inhabits this architecture. We write the program that will direct the agent behaviour, but much of what the agent effectively does is determined by the architecture itself, without the programmer having to worry about it. The language interpreted by *Jason* is an extension of AgentSpeak, which is based on the BDI architecture. Thus, one of the components of the agent architecture is a belief base, and an example of one of the things that the interpreter will be doing constantly, without it being explicitly programmed – even though this can be customised, which we shall see in later chapters – is to perceive the environment and update the belief base accordingly. Another important component is the agent's *goals*, which are achieved by the execution of *plans*.

Recall that practical BDI agents are *reactive planning systems*. These are reactive systems in the sense described previously: systems that are not meant to compute the value of a function and terminate, but rather designed to be permanently running, reacting to some form of 'events'. The way reactive planning systems react to such events is by executing plans; plans are courses of action that agents commit to execute so as to handle such events. The actions in turn change the agent's environment, in such a way that we can expect the agent's goals to be achieved. Again, there is a component of the agent architecture which will be responsible for effecting the changes in the environment based on the *choices* on the course of actions made by the interpretation of the agent program.

The interpretation of the agent program effectively determines the agent's *reasoning cycle*. An agent is constantly perceiving the environment, reasoning about how to act so as to achieve its goals, then acting so as to change the environment. The (practical) reasoning part of the agent's cyclic behaviour, in an AgentSpeak agent, is done according to the plans that the agent has in its *plan library*. Initially, this library is formed by the plans the programmer writes as an AgentSpeak program.

We shall see in detail all the components of the agent architecture and how the interpreter works in the next chapter. In this chapter, we describe all the language constructs that programmers can use, which can be separated into three main categories: beliefs, goals and plans. There are various different types of constructs used in each category; we will first introduce all of them and then give an example using as many language constructs as possible.

3.1 Beliefs

The very first thing to learn about AgentSpeak agents is how to represent the agents' beliefs. As we mentioned above, an agent has a *belief base*, which in its simplest form is a collection of literals, as in traditional logic programming. In programming languages inspired by logics, information is represented in symbolic form by *predicates* such as:

```
tall(john) .
```

which expresses a particular *property* – that of being *tall* – of an object or individual, in this case ‘*John*’ (the individual to whom the term `john` refers). To represent the fact that a certain relationship holds between two or more objects, we can use a *predicate* such as:

```
likes(john, music) .
```

which, pretty obviously, states that John likes music. A *literal* is such a predicate or its negation – we say more about negation below.

However, unlike in classical logic, we are here referring to the concept of *modalities of truth* as in the modal logic literature, rather than things that are stated to be true in absolute terms. When a formula such as `likes(john, music)` appears in an agent's belief base, that is only meant to express the fact that the agent currently *believes* that to be true; it might well be the case that John does not like music at all.

We shall not dwell too long on the issue of symbolic representation of logical statements; the vast literature on logic and logic programming [19, 28, 57, 63, 66, 92] is the best place to find more details about this. However, we introduce below some of the basic concepts, just so that the reader unfamiliar with a language such

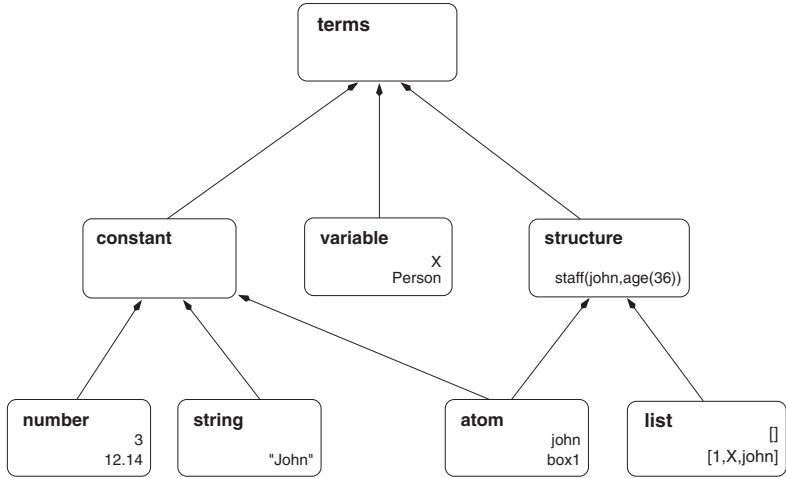


Figure 3.1 Types of AgentSpeak terms in *Jason*.

as Prolog can carry on reading this book. We concentrate here on presenting the main language constructs in logic programming, and Chapter 4 will give some examples which can help with understanding other things such as the use of logical variables and the important issue of *unification*. A summary of all the notions used to define the basic terms of the logic-based language used in this book is given in Figure 3.1.

Basics of logic programming

As in Prolog, any symbol (i.e. a sequence of characters) starting with a lower-case letter is called an *atom*, which is used to represent particular individuals or objects – they are the equivalent of ‘constants’ in first-order logic. For example, `john` is an atom (which can be used to represent a real person called John). A symbol starting with an uppercase letter is interpreted as a *logical variable*. So, while `john` is an atom, `Person` is a variable. Besides atoms, numbers and strings (which are represented as usual in programming languages) are also classified as *constants*.

Initially variables are *free* or *uninstantiated* and once *instantiated* or *bound* to a particular value, they maintain that value throughout their *scope* – in the case of AgentSpeak, we shall see that the scope is the *plan* in which they appear. Variables are bound to values by *unification*; a formula is called *ground* when it has no more uninstantiated variables. Using the examples above, a variable `Person` could be bound to the atom `john`. More generally, variables

can be bound to a constant (i.e. an atom, a number, or a string), or to a more complex data type called *structure*. The word *term* is used to refer to a constant, a variable or a structure.

A *structure* is used to represent complex data, for example `staff("Jomi Fred Hübner", 145236, lecturer, married, wife(ilze), kids([morgana, thales]), salary(133987.56))` could be used to represent the information about a member of staff in a university. Structures start with an atom (called the *functor*) and are followed by a number of terms (called *arguments*) separated by commas and enclosed in parentheses. Observing the predicates (called *facts* in Prolog) shown earlier to exemplify an agent's beliefs, we can see that they have exactly the same format as structures. The difference is purely semantical: a structure is used as term, to represent an individual or object, whereas a fact is used to represent a logical proposition – i.e. a formula that is to be interpreted as being either true or false. The number of arguments of a predicate/structure is important, and is called its *arity*. A particular structure is referred to by its functor and its arity: the structure above, for example, is referred to by 'staff/7', as it must have always exactly seven different terms as arguments (otherwise it is considered a different structure altogether).

A special type of structure was used in the example above: `[morgana, thales]`. Although internally this is represented as a structure, this special notation, called a *list*, is very useful in logic programming. There are special operations for lists; in particular ']' can be used to separate the first item in a list from the list of all remaining items in it. So, if we match (more precisely, unify) a list such as `[1, 2, 3]` with list `[H|T]`, `H` (the head) is instantiated with `1` and `T` (the tail) with the list `[2, 3]`. The Prolog syntax for lists is also used in *Jason*.

Figure 3.1 shows the classification of the various types of terms available in the language interpreted by *Jason*. Terms are then used to compose more complex types of formulae, as shown later in Figure 3.2.

Annotations

Perhaps one important difference in the syntactical representation of logical formulae in *Jason* compared with traditional logic programming is the use of *annotations*. These are complex terms (cf. Prolog structures) providing details that are strongly associated with one particular belief. Annotations are enclosed in square brackets immediately following a literal, for example:

```
busy(john) [expires(autumn)] .
```

which, presumably, means that the agent believes that John is busy, but as soon as autumn starts, that should be no longer believed to hold. Note that the annotation `expires(autumn)` is giving us further information about the `busy(john)` belief in particular. Readers familiar with logic will see that annotations of this form do not increase the expressive power of the language. We could equally well have written something like this:

```
busy_until(john, autumn).
```

to mean pretty much the same thing. Or if we wanted to have the possibility of marking down when each single belief is to be expired (and removed from the belief base), we could have used `annot(b, expires(...))` for each belief `b` that has a date to expire, where `annot` would be a special predicate symbol to be used by customisations of the *Jason* interpreter. However, there are two advantages of the way annotations are used in *Jason*. The first is simply the elegance of the notation. Elegance might be irrelevant logically, but for many programmers it is of great importance! The fact that the details about a particular belief are organised in such a way so as to be visually linked to belief itself makes the belief base considerably more readable. The second advantage is that this representation also facilitates management of the belief base. For example, the `expires` annotation is presumably meant to influence the duration of time that the `busy(john)` belief will be kept in the belief base.

Note that the `expires` annotation means absolutely nothing to the *Jason* interpreter. One should not expect *Jason* to delete that belief simply because the programmer annotated when the belief expires. On the other hand, it would be very easy for the programmer to customise, by doing a little (and very simple) Java programming, an agent's belief base (or indeed the agent's belief update function) so that the `busy(john)[expires(autumn)]` belief was automatically removed as soon as the agent believed `autumn` to be true. We shall see how this can be done in Chapter 7.

There are other belief annotations, however, which do have specific meanings for the interpreter, and very important ones, for that matter. In particular, the `source` annotation is used to record what was the source of the information leading to a particular belief. In fact, the `source` annotation is the reason why annotations were created in the first place, even though they later turned out to be useful for so many (application-specific) programming tasks.

There are three different types of information source for agents in multi-agent systems:

perceptual information: an agent acquires certain beliefs as a consequence of the (repeated) sensing of its environment – a symbolic representation of a property of the environment as perceived by an agent is called a *percept*;

communication: as agents communicate with other agents in a multi-agent system, they need to be able to represent the information they acquired from other agents, and it is often useful to know exactly which agent provided each piece of information;

mental notes: it makes certain programming tasks easier if agents are able to remind themselves of things that happened in the past, or things the agent has done or promised: beliefs of this type are added to the belief base by the agent itself as part of an executing plan, and are used to facilitate the execution of plans that might be selected at a later stage. The best way to think of this type of belief is as a ‘mental note’; that is, things the agent itself will need to remember in certain future circumstances.

One of the things that is perhaps different in *Jason* compared with other agent programming languages is the way *Jason* makes the source of information explicit in the belief base. For beliefs originating from perceptual information, the interpreter automatically adds an annotation `source(percept)`, which tells us that this particular belief was perceived by the agent itself rather than, for example, communicated. An annotation `source(self)` is used for mental notes (these are explained in more detail on page 52). Any other source of information annotated by the interpreter will be the name of the agent which sent a communication message that provided the information leading to that belief. For example, `likes(john,music) [source(john)]` says that our agent believes `john` likes music because agent `john` itself informed our agent of this (which implies that, at least at the time that belief was acquired, there was an agent named `john` which was a member of the same *society* as our agent).

Strong Negation

Negation is the source of many difficulties in logic programming languages. A popular approach to handling negation in logic programming is to use the *closed world assumption* and *negation as failure*. The closed world assumption may be understood as follows:

Anything that is neither known to be true, nor derivable from the known facts using the rules in the program, is assumed to be false.

The only type of negation in this context, given by the ‘not’ operator, is such that the negation of a formula is true if the interpreter *fails* to derive the formula using the facts and rules in the program (rules are also available in our language, as we see later). Another type of negation, denoted here by the ‘~’ operator, and called *strong negation*, is used to express that an agent *explicitly believes something*

to be false. For example, the belief `colour(box1,white)` means that the agent believes the colour of `box1` is white, whereas the belief `~colour(box1,white)` means that the agent believes that it is *not* the case that the colour of `box1` is white.

Advanced

Under the closed world assumption, anything not explicitly believed to be true by the agent is believed to be false. However, this assumption is too simple and too strong for many applications. For open systems (such as, e.g., the Web), it is significantly easier to produce a model for an agent's beliefs if one is able to refer to the things the agent believes to be true, the things the agent explicitly believes to be false, as well as the things the agent is ignorant about. For this, it helps to have a different type of negation, which here is denoted by ' $\sim$ '. So, if `not p` and `not ~p` are true (meaning that neither `p` nor `~p` are in – or can be derived from – the agent's belief base) we can conclude that the agent has no information about whether `p` is true or not. We postpone giving a more concrete example for this until we have seen how we write plans in AgentSpeak.

Recall that some of the beliefs in an agent's belief base will have been placed there by the agent repeatedly sensing the environment. Therefore, if the closed world assumption is to be used, make sure that the program does not use strong negation and also make sure that the percepts the agent obtains when sensing the environment (i.e. by its perception function) only contain *positive literals*; that is, strong negation cannot be used to model the environment either. Chapter 5 will discuss simulated environments, and Section 7.3 will discuss the customisation of the perception function.

Now, here is an example which includes all we have learnt about the belief base so far. Consider the following excerpt of `maria`'s belief base:

```
colour(box1,blue) [source(bob)] .
~colour(box1,white) [source(john)] .
colour(box1,red) [source(percept)] .
colourblind(bob) [source(self),degOfCert(0.7)] .
liar(bob) [source(self),degOfCert(0.2)] .
```

The example tells us that agent `maria`, whose belief base is as above, was informed by agent `bob` that a particular box named `box1` is blue (this is what the first line of the example above says) and was informed by agent `john` that the box is *not* white (second line of the example). However, agent `maria` (later) perceived – e.g. by means of a camera – that `box1` is red, according to the third line of the example. Now, normally, and in particular in open systems where agents cannot be assumed to be trustworthy, perceptual information is more reliable than communicated

information, so in principle this agent can conclude that either `bob` is colourblind, and therefore was genuinely mistaken about the colour of `box1`, or else that `bob` intentionally tried to deceive `maria`. Comparing the degrees of certainty (annotation `degOfCert`) that have been associated with the last two lines of the example, we can imagine that `maria` has had some form of interaction with `bob` and is fairly confident of its trustworthiness, given that `maria` seems to be much more certain of the possibility that `bob` is colourblind than of it being a liar. Note the `source(self)` annotations in the last two lines, which tell us that those beliefs were added to the belief base by the agent itself (how an agent can add such derived beliefs to the belief base will be seen below on page 52). Again, we should emphasise that `degOfCert` means nothing to the interpreter, that is to say that any operations on, or considerations of, degrees of certainty need to be taken care of by the programmer rather than to expect the interpreter to do this automatically.

Nested annotations

Advanced

It is also possible to use *nested annotations*. For example, in `bob`'s belief `loves(maria,bob) [source(john) [source(maria)]]` there is an annotation (`source(maria)`) to `source(john)` which is itself an annotation. This would happen, for example, if agent `john` sent a message to `bob` with content `loves(maria,bob) [source(maria)]`. What this represents is that agent `john` is telling `bob` that `maria` told him that she loves `bob`. Note further that, if the multi-agent system assumes the principle of *veracity* (see [103]), meaning that the designer can assume that all agents in the system only inform other agents of things they themselves believe to be true, then `loves(maria,bob) [source(john) [source(maria)]]` could also represent a nested belief modality (for the reader interested in modal logics): after this communication, `bob` believes that `john` believes that `maria` believes that she loves `bob`.

Rules

Readers with experience of Prolog will be familiar with the idea of rules, which allow us to conclude new things based on things we already know. Including such rules in an agent's belief base can simplify certain tasks, for example in making certain conditions used in plans (as discussed in Section 3.3 below) more succinct. However, for simple applications or for those who are not Prolog experts, it may be preferable not to use rules altogether. If they are used, Prologers will note that the notation is slightly different, the reason being that the types of formulæ that can

appear in the body of the rules also appear elsewhere in the AgentSpeak language (in a plan ‘context’), so we use that same notation here for the sake of homogeneity.

Consider the following rules:

```
likely_colour(C,B)
:- colour(C,B)[source(S)] & (S == self | S == percept).

likely_colour(C,B)
:- colour(C,B)[degOfCert(D1)] &
   not (colour(_,B)[degOfCert(D2)] & D2 > D1) &
   not ~colour(C,B).
```

The first rule says that the most likely colour of a box is either that which the agent deduced earlier, or the one it has perceived. If this fails, then the likely colour should be the one with the highest degree of certainty associated with it, provided there is no strong evidence the colour of the box is *not* that one. To the left of the ‘:-’ operator, there can be only one literal, which is the conclusion to be made if the condition to the right is satisfied (according to the agent’s current beliefs).

The type of reasoning that the agent is doing in the example above is referred to in the literature as *theoretical reasoning*; that is, reasoning over an abstract representation of things known in order to derive further knowledge. We mentioned in the initial chapters that the underlying model of decision-making in BDI agents is called *practical reasoning* (i.e. reasoning directed towards actions); in AgentSpeak, this is captured by the agent’s *plans*, as defined in Section 3.3.

Although AgentSpeak programmers will manipulate and inspect the belief base in various ways, in AgentSpeak, all that has to be provided are the *initial beliefs* (and rules, if any). That is, the source code tells the interpreter which beliefs should be in the agent’s belief base when the agent first starts running, even before the agent senses the environment for the very first time. Unless specified otherwise, all beliefs that appear in an agent’s source code are assumed to be ‘mental notes’. Thus, if we have `likes(john, music)` in the source code, when inspecting the agent’s belief base we will note that the belief is in fact represented as `likes(john, music)[source(self)]`; we mention in Section 4.3 how to ‘inspect’ an agent’s mental state in *Jason*. If by any chance the agent has to have an initial belief that is to be treated as if it had been acquired, e.g. by sensing the environment, that needs to be made explicit, for example as in `tall(john)[source(percept)]`. We shall see in Sections 4.1 and 7.2 how different types of beliefs are treated differently when belief update/revision takes place.

We now turn from belief to another key notion in AgentSpeak programming: that of a *goal*.

3.2 Goals

In agent programming, the notion of *goal* is fundamental. Indeed, many think this is the essential, defining characteristic of agents as a programming paradigm. Whereas beliefs, in particular those of a perceptual source, express properties that are believed to be true of the world in which the agent is *situated*, goals express the properties of the states of the world that the agent *wishes to bring about*. Although this is not enforced by the AgentSpeak language, normally, when representing a goal g in an agent program, this means that the agent is committed to act so as to change the world to a state in which the agent will, by sensing its environment, *believe* that g is indeed true. This particular use of goals is referred to in the agent programming literature as a *declarative goal*.

In AgentSpeak, there are two types of goals: *achievement goals* and *test goals*. The type of goal described above is (a particular use of) an achievement goal, which is denoted by the ‘!’ operator. So, for example, we can say `!own(house)` to mean that the agent has the goal of achieving a certain state of affairs in which the agent will believe it owns a house, which probably implies that the agent does not currently believe that `own(house)` is true. As we shall see in the next section, the fact that an agent may adopt a new goal leads to the execution of *plans*, which are essentially courses of actions the agent expects will bring about the achievement of that goal.

Perhaps an important distinction to make is with the notion of ‘goal’ as used in Prolog, which is rather different from the notion of goal used above. A goal or ‘goal clause’ in Prolog is a conjunction of literals that we want the Prolog interpreter to check whether they can be concluded from the knowledge represented in the program; the Prolog interpreter is essentially proving that the clause is a logical consequence of the program. This is also something that we will need doing as part of our agents – that is, checking whether the agent believes a literal or conjunction of literals – and this is why there is another type of goal in AgentSpeak, called a test goal and denoted by the ‘?’ operator. Normally, test goals are used simply to retrieve information that is available in the agent’s belief base. Therefore, when we write `?bank_balance(BB)`, that is typically because we want the logical variable `BB` to be instantiated with the specific amount of money the agent currently *believes* its bank balance is.

More specifically, the agent could have a belief `bank_balance(100)`, in which case it is trivial to find a unification for the test goal (i.e. where $BB \mapsto 100$), as one would expect in Prolog. Also as in Prolog, we may need to use rules in the belief base to *infer* an answer to our test goal. Furthermore, in the next section we shall see that, as for achievement goals, test goals can also lead to the execution of plans in certain circumstances.

Procedural and declarative goals

Achievement goals can be used *procedurally* or *declaratively*. In a *procedural goal*, the goal name is similar to the name of a method/procedure in traditional programming languages, which of course can be quite useful (e.g. to group together actions that are often used in the program). However, what makes the use of goals special in agent programming is when we introduce the notion of a *declarative goal*, which we already briefly mentioned above. The importance of this notion is that goals are then seen as a symbolic representation of a state of affairs that the agent has to achieve, and if an agent actually has such an explicit representation, it can then check whether having executed a piece of code (i.e. a plan) has effected, in regards to the changes in the environment, the expected result. Because multi-agent systems are particularly suitable for complex, dynamic environments, being able to make such checks is absolutely essential. Further, having such a representation means that an agent can be more naturally programmed to actively persist in attempting to achieve its goals, until the agent believes the goal has been effectively achieved. We shall return to the issue of declarative goals, and various forms of commitments that agents can have towards such goals, in Chapter 8.

In the same way that, in the actual source code, we only provide the beliefs that we wish the agent to have at the moment it first starts running, we can also provide goals that the agent will attempt to achieve from the start, if any such goals are necessary for a particular agent. That is, we can provide *initial goals* to our agent. For example, this is an agent that would start running with the long term goal of getting the house clean.

```
!clean(house) . . .
```

Beliefs and goals are the two important mental attitudes that we can express in an agent source code. The third essential construct of an agent program, in the BDI style, are *plans* – the agent’s know-how – and the way plans are executed is also important. It is precisely the *changes* in the agent’s *beliefs* and *goals* that trigger the execution of plans. So next we see in detail how plans are written.

3.3 Plans

An AgentSpeak plan has three distinct parts: the *triggering event*, the *context*, and the *body*. Together, the triggering event and the context are called the *head*

of the plan. The three plan parts are syntactically separated by ‘:’ and ‘<-’ as follows:

```
triggering_event : context <- body.
```

We shall explain in detail each of these parts in turn, but first we briefly describe what is the idea behind each part of a plan:

Triggering event. As we saw in the initial chapters, there are two important aspects of agent behaviour: reactivity and pro-activity. Agents have goals which they try to achieve in the long term, determining the agent’s pro-active behaviour. However, while acting so as to achieve their goals, agents need to be attentive to changes in their environment, because those changes can determine whether agents will be effective in achieving their goals and indeed how efficient they will be in doing so. More generally, changes in the environment can also mean that there are new opportunities for the agent to do things, and perhaps opportunities for considering adopting new goals that they previously did not have or indeed dropping existing goals.

There are, accordingly, two types of changes in an agent’s mental attitudes which are important in an agent program: changes in beliefs (which, recall, can refer to the information agents have about their environment or other agents) and changes in the agent’s goals. Changes in both types of attitudes create, within the agent’s architecture, the *events* upon which agents will act. Further, such changes can be of two types: *addition* and *deletion*. As we mentioned in the beginning of this chapter, plans are courses of actions that agents commit to execute as a consequence of such changes (i.e. events).

The triggering event part of a plan exists precisely to tell the agent, for each of the plans in their plan library, which are the specific events for which the plan is to be used. If an event that took place matches – and we describe in detail how that matching happens in the next chapter – the triggering event of a plan, that plan might start to execute, provided some conditions are satisfied, as we see next. If the triggering event of a plan matches a particular event, we say that the plan is *relevant* for that particular event.

Context. As for triggering events, the *context* of a plan also relates to an important aspect of reactive planning systems. We have seen that agents have goals, and plans are used to achieve them, but also that agents have to be attentive to changes in the environment. Dynamic environments are complicated to deal with first because changes in the

environment may mean we have to act further, but also because, as the environment changes, the plans that are more likely to succeed in achieving a particular goal also change.

This is why reactive planning systems postpone committing to courses of action (i.e. a plan) so as to achieve a particular goal until as late as possible; that is, the choice of plan for one of the many goals an agent has is only made when the agent is about to start acting upon it. Typically, an agent will have various different plans to achieve the same goal. (In addition, various plans for different goals can be competing for the agent's attention, but we leave this issue for the next chapter.)

The context of a plan is used precisely for checking the current situation so as to determine whether a particular plan, among the various alternative ones, is likely to succeed in handling the event (e.g. achieving a goal), given the latest information the agent has about its environment. Therefore, a plan is only chosen for execution if its context is a *logical consequence* – and we discuss later exactly what that means – of the agent's beliefs. A plan that has a context which evaluates as true given the agent's current beliefs is said to be *applicable* at that moment in time, and is a candidate for execution.

Body. The body of a plan, in general terms, is the easiest part of a plan to understand. This is simply a sequence of formulae determining a course of action – one that will, hopefully, succeed in handling the event that triggered the plan. However, each formula in the body is not necessarily a 'straight' action to be performed by the agent's effectors (so as to change the environment). Another important construct appearing in plan bodies is that of a *goal* this allows us to say what are the (sub)goals that the agent should adopt and that need to be achieved in order for that plan to handle an event successfully. We can refer to the term *subgoals*, given that the plan where they appear can itself be a plan to achieve a particular goal – recall that the triggering events allow us to write plans to be executed when the agent has a new goal to achieve. In fact, there are other things that can appear in the body of a plan, but we see this in detail below.

Next, we introduce how exactly each part of the plan is written in an agent program.

Triggering Events

As we saw earlier, there are two types of mental attitudes: beliefs and goals. Further, there are two different types of goals: 'test' and 'achievement'. Events represent

Table 3.1 Types of triggering events.

Notation	Name
$+l$	Belief addition
$-l$	Belief deletion
$+!l$	Achievement-goal addition
$-!l$	Achievement-goal deletion
$+?l$	Test-goal addition
$-?l$	Test-goal deletion

Nomenclature for the six types of triggering events that plans can have.

changes in beliefs and goals, and the changes can be of two types: ‘addition’ or ‘deletion’. These allows us to express six different types of triggering events for a given literal l , with notation and name as shown in Table 3.1.

Events for belief additions and deletions happen, for example, when the agent updates its beliefs according to its perception of the environment obtained (normally) at every reasoning cycle. Recall that the reasoning cycle and the handling of events will be discussed in detail in the next chapter.

Events due to the agent having new goals (i.e. goal additions) happen mostly as a consequence of the execution of other plans, but also as a consequence of agent communication, as we shall see in Chapter 6. When we discuss the plan body below, we shall see that executing a plan may require the agent having new goals to achieve/test. The goal deletion types of events are used for handling plan failure; we defer discussing this until Section 4.2, but keep in mind that these six types of triggering events exist.

Figure 3.2 shows how simpler formulæ are used to form the various types of AgentSpeak formulæ that have been described above (refer back to Figure 3.1 for a diagram with the various types of terms used to build formulæ).

To make the use of triggering events clearer, consider the following examples. Suppose we want to develop an agent that may need to have the goal of serving dinner, which implies that the goal will have been achieved when the agent believes that dinner is served, represented as `served(dinner)`. We will need a plan that the agent will start executing when it comes to have the goal of achieving a state of affairs where the agent will (hopefully correctly) have such a belief; for this, the plan will have a triggering event such as `+!served(dinner)`. Or, more generally, the agent might have to serve different kinds of meals, and in this case the triggering event would probably look like `+!served(Meal)` so that, if one particular event is that the agent has a new goal to have *dinner* served, then the plan would be executed with the logical variable `Meal` instantiated with `dinner`.

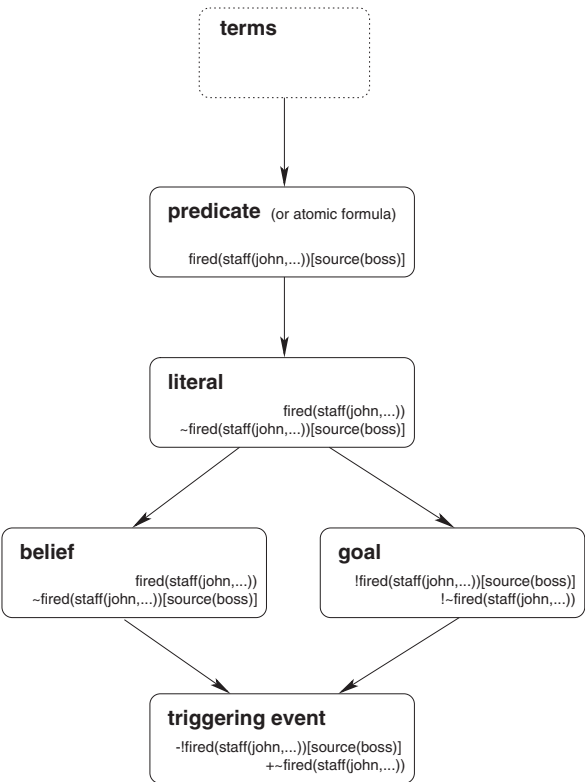


Figure 3.2 Types of AgentSpeak formulae.

With regards to test goals, they are normally used to retrieve simple information from the belief base. For example, I normally remember how many packets of pasta I have in the larder, so when I need that information, I just retrieve it from memory. However, if I happen not to have that information to hand, I may need to perform an action specifically to find out this information. The action might be to ask someone else, or to go to the larder myself. This is why plans with test-goal addition as triggering events exist, even though with extra computing steps (and less transparency) this could be solved with achievement goals. (We will see later how this can be done.)

Advanced

Plans with test goal addition in the triggering events exist because in some cases it might be useful to be able to write plans to retrieve information in a more elaborate way, by taking some action. When the agent has a test goal, the interpreter will first try to obtain the result for it by simply using facts (and possibly rules) in the belief base, but if that fails, then an event is generated so

that a plan for that test goal might be executed, if possible. So, for the example above, we could write a plan with a triggering event `+?stock(pasta,X)` in order for the agent to obtain this information in case it happens not to be available in the agent’s belief base when the information is needed for the execution of another plan.

Context

Recall that contexts are used to define when a plan should be considered *applicable*. Recall also that, because we are writing programs for dynamic environments, the idea is to postpone committing to a course of action until as late as possible. So, when it comes to choosing a plan – a course of action – in order to achieve a goal, it is the context that will be used to ensure that we are choosing a plan that is likely to succeed.

The way we write a plan context is as follows. It is often the case that a context is simply a conjunction of *default literals* and relational expressions – the syntax for expressions is very similar to Prolog, and also discussed below as part of the plan body. Recall that a literal is a predicate about some state of the world, which may have a strong negation. A default literal is a literal which may optionally have another type of negation, known in logic programming as *default negation*, which is denoted by the ‘**not**’ operator. This is effectively used to check that a literal (with a strong negation or not) is not currently in the belief base. This gives us the four combinations of literals in Table 3.2. Logical expressions can appear in a plan context by combining, in the usual way, literals with the operators *and* (conjunction) denoted by ‘&’ and *or* (disjunction) denoted by ‘|’ as well as negation (the negation as failure operator ‘**not**’). This exact same syntax can be used in the body of the rules that are part of the belief base.

Note that, for example, not believing that *l* is false (i.e. not believing *~l*) is not the same as believing *l* is true, because the agent might simply be ignorant about *l* (i.e. the agent does not know anything about *l* at all). This may sound a bit confusing, so a good thing to keep in mind is that, for those not already

Table 3.2 Types of literals in a plan context.

Syntax	Meaning
<i>l</i>	The agent believes <i>l</i> is true
<i>~l</i>	The agent believes <i>l</i> is false
not <i>l</i>	The agent does not believe <i>l</i> is true
not <i>~l</i>	The agent does not believe <i>l</i> is false

A plan context is typically a conjunction of these types of literals.

familiar with these notions, or for applications where it seems it might not be really necessary to use this kind of representation, it may be better to use good old closed-world assumption instead, and simply avoid using strong negation altogether.

According to what we have seen so far, examples of plan contexts could be as follows:

```
+!prepare(Something)
  :  number_of_people(N) & stock(Something,S) & S > N
  <-  ... .

...

+!buy(Something)
  :  not ~legal(Something) & price(Something,P)
    & bank_balance(B) & B > P
  <-  ... .
```

That is, when the agent comes to have the goal of preparing something (to eat), provided it has enough of it in stock for the number of people who will be eating, the first plan can be used to handle that event (i.e. the ‘event’ that occurred because there is a new goal to achieve). As we have not yet seen what the body of a plan looks like, we gave just plan sketches above, hence the ‘...’ to denote we have omitted part of the plan.

The second outline plan above says that, when the agent comes to have a new goal to buy something, provided it does not have explicit information saying it is illegal to buy this thing, and it has enough money for the purchase, that plan might be eligible for execution. Another example using strong negation is given later, after we see the things we can write in a plan body.

One important thing to bear in mind is that any literal appearing in the context is to be checked against the agent’s *belief base*. For example, if `stock(pasta,5)` is in the belief base, the result of literal `stock(Something,S)` appearing in a plan context, assuming variable `Something` is already bound to `pasta`, will evaluate to true with further instantiation of the plan variables, in particular `S` will become bound to 5. This is generalised to checking whether the whole context is a logical consequence of the belief base.

Advanced

For the sake of completeness, it should be noted that the context can also have formulæ that are not to be checked against the agent’s beliefs. In particular, there can be ‘internal actions’ which can be used, for example, to execute Java code on the partial results already obtained in evaluating the context. Internal

actions can also appear in plan bodies, and their use in plan contexts is exactly the same. Therefore, we leave the use of internal actions to be explained in detail below, with the various other things that can appear in a plan body. However, at this point it is worth mentioning that programmers should be careful with internal actions in plan contexts, as they are executed *in order to check if the plan is applicable*; this means that, if a relevant event happens, those actions might be executed when the interpreter is determining the set of applicable plans, hence executed even if that plan turns out not to be the one selected to handle that event. Needless to say, an agent is likely to become very inefficient if heavy code is carelessly used in plan contexts.

The rules in the belief base can be used to make plan contexts much more compact; for example, instead of:

```
+!buy(Something)
  : not ~legal(Something) & price(Something,P)
    & bank_balance(B) & B>P
  <- ... .
```

we could add

```
can_afford(Something)
  :- price(Something,P) & bank_balance(B) & B>P.
```

to the belief base and then write the plan as:

```
+!buy(Something)
  : not ~legal(Something) & can_afford(Something)
  <- ... .
```

We now turn to the last part of a plan, its body.

Body

The body of a plan defines a course of action for the agent to take when an event that matches the plan's triggering event has happened and the context of the plan is true in accordance with the agent's beliefs (and the plan is chosen for execution). The course of action is represented by a sequence of formulæ, each separated from the other by '*;*'. There are six different types of formulæ that can appear in a plan body, and we now see each of them in turn.

Actions

One of the most important characteristics of an agent is that it must be able to *act* within an environment. Thus, unsurprisingly, a type of formula found in most plans is *actions*, which represent the things the agent is capable of doing. If we are programming a robot, we will know the actions the agent is capable of doing (those its hardware allow it to do). We will then need symbolic representations of these actions – which is how we will refer to them in the program – and then the overall agent architecture will interface these with the action effectors (i.e. the physical or software mechanism that executes the actions). We discuss this interface in Section 7.3, but for now, we only need to worry about the symbolic representation that will be used in the program.

The language construct for symbolically referring to such actions is simply a *ground predicate*. That is, we need to make sure that any variables used in the action become instantiated before the action is executed. One may wonder how the interpreter can differentiate actions from other uses of predicates. This is done simply by the position within the plan. For example, predicates in the context of the plan will be checked against the agent's beliefs, rather than being seen as actions. In the body of the plan, a simple predicate will be interpreted as an action: it will be clear below that all other types of formulæ that can appear in a plan body also have a predicate (or literal) involved but there are special notations to make it clear it is not an environment-changing action.

We will later see that there is another type of construct in the plan body called 'internal action' which, unlike the actions we are defining here, run internally within an agent rather than change the environment. We already saw an example of an internal action earlier: `.print(...)`, used to display text on a programmer's console.

As a simple example, suppose we are writing a program for a robot that has two arms ('right' and 'left') and the only action it can do is to rotate those arms – a rotation angle has to be specified. We could then have, in our plans, an action such as `rotate(left_arm, 45)`, or we could choose to represent the actions as, say, `rotate_right_arm(90)` instead. Which we should choose is partly a matter of personal taste, and partly depends on the problem at hand.

Actions are executed by an agent's effectors, which are not part of the AgentSpeak interpreter. Therefore it makes sense to expect some kind of 'feedback' on whether the action was executed or not: it may turn out that it was impossible to carry out the action. Until such (Boolean) feedback is received, the plan that required the action to be executed must be *suspended* until we know the action has been executed. Executing an action does not mean that the expected changes will

necessarily take place – this needs to be confirmed by perceiving the environment and checking subsequent beliefs. All the ‘feedback’ tells us is whether the requested action was executed at all; if it was not, the plan fails. We discuss this in more detail in the next chapter.

Achievement Goals

Having goals to achieve is another essential characteristic of an agent. Complex behaviour requires more than simple sequences of actions to execute; it involves achieving goals before further action can be taken. This is why we need to be able to say, as part of a plan body, that a goal has to be achieved, and only then the rest of the plan can be executed. As with all constructs we have seen so far, the representation will involve a literal; the way an achievement goal to be added to the agent’s current goals is denoted is by prefixing the literal with the ‘!’ operator. So, for example, we can say that the agent should have, as part of a course of action, the goal of achieving a state of affairs in which the customer is happy; for that, we could include in such plan body something like `!happy(customer)`, or to achieve a state of affairs in which the agent believes that there is no (longer any) gas leak, we could use `!~leaking(gas)`.

As we know from the types of triggering events we already saw, having a new goal is also considered an ‘event’ that can lead to the execution of a plan. This means that, if we include a goal in a plan body, a plan needs to be executed for that goal to be achieved, and only after the plan has successfully executed can the current course of action be resumed; so here again, the plan will be *suspended* (more on this in the next chapter). To be clearer, if we have ‘`a1; !g2; a3`’ in a plan body, the agent will do `a1`, then it will have the goal of achieving `g2`, which will involve some other plan being executed, and only when that plan is finished can `a3` be executed. In fact, to be able to program proficiently in AgentSpeak, it helps to have a more detailed understanding of how exactly the interpreter works. This will be discussed in detail in Chapter 4.

A plan body is a *sequence* of formulæ to be executed. At times, it may be the case that we want the agent to have another goal to achieve but we do not need to wait until this has actually been achieved before we can carry on with a current course of action. The way to express this is to use the ‘!!’ operator. In more advanced examples, we will see that this is also useful to make recursive plans more efficient. For the time being, just consider that, if the agent has ‘`!at(home); call(john)`’ as part of a plan, it will only call John when it gets home, whereas if it is ‘`!!at(home); call(john)`’, the agent will call John soon after it comes to have a new goal of being at home, possibly even before it does any action in order to achieve that goal.

Test Goals

Test goals are normally used to retrieve information from the belief base, or to check if something we expected is indeed believed by the agent, during the execution of a plan body. One may wonder why we do not use the context of the plan to get all the necessary information before the agent starts on its course of action. Conceptually, the context should be used only to determine if the plan is applicable or not (i.e. whether the course of action has a chance of succeeding), and practically we may need to get the latest information which might only be available, or indeed might be more recently updated, when it is really necessary. For example, if the agent is guaranteed to have some information about a target's coordinates, which are constantly updated, and only needed before a last action in the course of action defined by the plan body, then it would make more sense to use `?coords(Target, X, Y) ;` just before that action than to include `coords(Target, X, Y)` in the context. In this particular example of test goal, presumably the variable `Target` would already be bound (by previous formulæ in the body) with the particular target in question, and assuming `x` and `y` were free (i.e. not bound yet), any use of those variables after the test goal will be referring to the particular coordinates – as believed at the time the test goal is executed – of the target referred to by `Target`.

In the next chapter, we shall see that an agent's intentions are stacks of *partially instantiated plans*. This notion of instantiating variables with specific values as late as possible, as well as the late choice of a particular plan to achieve a goal, are very important for agent programming, precisely because of the dynamic nature of the environment.

As we mentioned in the description of triggering events, test goals can also be used for more complex tasks than just retrieving information already available in the belief base. Whilst we want to leave the instantiation of variables for as late as possible, there is an important issue with test goals. If the context of a plan does not follow from the agent's beliefs, the plan is not even selected to be executed. However, if we try to execute a plan which has a test goal in it and the test goal fails, then the whole plan where the test goal appeared fails, and the mechanism for handling plan failure will then be called upon. Therefore, care must be taken in the use of test goals.

This is partly the reason why creating plans with test-goal additions as triggering events might be interesting. Suppose we expect the agent to have some information while the plan is executed, so we introduce the appropriate test goal in the plan. If for some unanticipated reason the agent happens not to have the information required for the plan to be executed further, this would normally simply fail the plan. Instead, what happens in *Jason* is that, before failing, the interpreter

first tries to create an event, which could match with a ‘test-goal addition’ triggering event in case the programmer created a plan to circumvent such situation; only in the case that *no* such (relevant) plan is available is the test goal effectively considered failed.

As a final note, consider that we could use an achievement goal to obtain the information required in the plan, as achievement goals can also be called with uninstantiated variables which become instantiated when the goal is achieved (i.e. a plan for it finishes executing). However, this forces the generation of an event for handling the achievement-goal addition, whereas a test goal normally (i.e. if the information is already in the belief base) just further instantiates the variables in the plan and the plan execution can be immediately resumed (if selected again in the next reasoning cycle, as we see in the next chapter). Note that, as we shall see in the next chapter, generating an event involves suspending the plan execution, finding a relevant plan, and so forth; this is just to emphasise that generating an event is rather expensive computationally, so it makes sense to have a specific test-goal construct.

Mental Notes

One important consideration when programming in *Jason* is to be aware of the things the agent will be able to perceive from the environment. Readers might be tempted to update an agent’s beliefs about the consequences of its actions as part of the agent program, as they would in other action-based programming languages. This is *not* to be done in *Jason*. In *Jason*, some beliefs are acquired by perceiving the environment and those are updated automatically; we see in Section 7.2 how we can customise belief updates.

On the other hand, in practice it is often useful to be able to create beliefs during the execution of a plan. We call these beliefs *mental notes*, and they have a `source(self)` annotation to distinguish them within the belief base. An agent may need a mental note to remind itself of something it (or some other agent) has done in the past, or to have information about an interrupted task that may need to be resumed later, or a promise/commitment it has made. For example, consider an agent that needs to remember when the lawn was last mowed; this is done in the plan body with a formula `+mowed(lawn,Today)`; or something similar. Recall that variable `Today` must have been instantiated previous to that. Then, when that formula appears in a plan being executed, a formula such as `mowed(lawn,date(2006,7,30))[source(self)]` will be added to the belief base.

A downside of this is that we normally have to explicitly delete mental notes that are no longer necessary, with a formula such as `-needs_be_done(mowing(lawn))`. The last operator in this category is available to simplify keeping the

belief base tidy. Because in some cases we only need the last instance of a certain mental note in the belief base, the operator `-+` can be used to remove a former instance (if any exist) while adding the new one. For example, we could use `++current_targets(NumTargets);` to keep only the current number of known targets (as instantiated to the `NumTargets` variable) and delete the information about what was previously believed to be the number of targets, in case any information about it was already in the belief base – note that attempting to delete a belief that does not exist is simply ignored, it does not cause a plan failure. Effectively, `++current_targets(NumTargets);` is simply an abbreviation for `-current_targets(_); +current_targets(NumTargets);` where, as in Prolog, `_` is the anonymous variable (i.e. it can unify with any value, and is useful where the value is irrelevant for the rest of the plan).

If necessary, it is also possible to change beliefs that have another source rather than `self`, but care should be taken in doing so because this is rather unusual. In case it is necessary, one could always write `-trustworthy(ag2)[source(ag1)];` – which means that the programmer wants the agent to forget that agent `ag1` informed their agent that `ag2` is a trustworthy agent – or `+turned_on(tv)[source(percept)]` if the programmer insists in ‘messing about’ with the agent’s perception of the environment.

Internal Actions

The first type of formula we saw were ‘actions’. The important characteristic of such actions is that they *change the environment*; we can think of them as being executed *outside* the agent’s mind (i.e. the agent’s reasoner). It is often useful to think of some types of processing that the agent may need to do also as if it was an action being requested by the reasoner, except that it is executed within the agent; the whole processing of the action will be done as one step of the agent’s reasoning cycle. This can be used by programmers to extend the programming language with operations that are not otherwise available, as well as providing easy and elegant access to legacy code – either in Java or other languages via the use of JNI (Java Native Interface).

The way an internal action is distinguished from (environment) actions is by having a ‘.’ character within the action name. In fact, the ‘.’ character is also used to separate the name of a library from the name of an individual internal action within that library (a Java ‘package’). Suppose we are using *Jason* to program a controller for a robot, and we have a library for the digital image processing that the robot will require on the images captured by a camera it uses to sense the environment. The library is called `dip` and within it we have an internal action for finding a path (called `get_path`) to given coordinates from the robot’s current location, in the last image taken by the robot’s camera. In this case, we could

have, in some plan body, a formula such as `dip.get_path(X,Y,P)`; and variable `P` would be instantiated by the internal action itself, with the path it found for the robot to move to (x,y) coordinates. We explain how we can create such internal actions in Section 7.1.

Besides being able to create internal actions, it is very important that programmers familiarise themselves with *Jason's standard internal actions* (a complete description of all currently available standard internal actions is given in Appendix A.3). These are internal actions that are distributed with *Jason*, and implement operations that can be very important for general BDI programming. A standard internal action is denoted by an empty library name; that is, the action symbol *starts* with the ‘.’ operator. One particular standard action to bear in mind already is the `.send` action, which is used for inter-agent communication. Using it properly involves some knowledge of agent communication languages, which were described in Section 2.5, and the particular communication language used in *Jason* that we only see in detail in Chapter 6. For the time being, consider that this action requires (at least) three parameters: the name of the agent that should receive the message, the ‘intention’ that the sender has in sending the message (e.g. to inform, to request information, to delegate goals) called the *performative*, and the actual message content.

We can now show a simple example using some of the things we have already seen about plans.

```
+!leave(home)
  : not raining & not ~raining
  <- !location(window);
    ?curtain_type(Curtains);
    open(Curtains);
    ...

+!leave(home)
  : not raining & not ~raining
  <- .send(mum,askIf,raining);
  ...
```

The first plan says that if the agent comes to have a new goal of leaving home in a circumstance such that it is not sure whether it is raining or not – note that the context of the plan says that the plan can only be used if the agent does *not* believe it is raining but does *not* believe it is *not* raining either. In that circumstance, one possible course of action to take is have the goal of being located at the window (a plan to achieve this would require the agent to physically move in the environment,

if it was not already there), then retrieve from the belief base the particular type of curtains that the agent is now capable of perceiving, so that it can next open that particular type of curtains (presumably, different types of curtain require different types of action to get them open). The plan could then carry on, possibly by having the goal of checking whether an umbrella needs to be taken – we omit the rest of the plan for simplicity. Before the plan carries on, in the reasoning cycle after the `open(Curtains)` action was executed, the agent will have sensed the environment and therefore will have updated its own beliefs about whether it is raining or not.

The second plan offers an alternative course of action for exactly the same situation: having the goal of leaving home and not knowing whether it is raining or not. Instead of going to the window and opening the curtains, we simply ask the ‘mum’ agent, who (always) knows whether it is raining or not. Which plan is actually chosen for execution depends on the agent. (A ‘teenager agent’ will have the know-how defined by both plans, but in all likelihood would choose the second plan – possibly shouting rudely, but that is slightly more difficult to model here.) In practice, the choice of which *applicable plan* to use is done by one of the *selection functions*; this is explained in Chapter 4.

Expressions

Each formula in the context and body of a plan must have a boolean value (including internal actions). As in the short examples we saw when discussing plan contexts, it is often the case that we may need to use relational expressions, which also have boolean values. They can also be used in plan bodies, and the syntax is very similar to Prolog. For example, we can include in a plan context (resp. body) ‘`x >= y*2;`’ so that the plan is only applicable (resp. can only carry on executing) if this condition is satisfied. Note that, in a plan body, the plan will fail, in a similar way to when an action fails, if the condition is not satisfied.

The operators that can be used are listed in the grammar in Appendix A.1, and they are inspired by those available in Prolog. For readers not familiar with Prolog, note that `==` and `\==` are used for ‘equal’ and ‘different’, respectively – two terms need to be exactly the same for `==` to succeed. Another useful relational operator in logic programming is whether two terms can be *unified*; the `=` operator is used for this. Arithmetic expressions can be used in the usual way, both within relational expressions and within literals. One operator also available in Prolog that works slightly differently here (because of the predicate annotations not available in Prolog) is ‘`=..`’, which is used to deconstruct a literal into a list. The resulting list has the format `[<functor>, <list of arguments>, <list of annotations>]`, for example:

```
p(b,c) [a1,a2] =.. [ p, [b,c], [a1,a2]].
```

Annotations in triggering events and context

As we saw earlier, in *Jason* literals can have annotations, and literals appear in the triggering event and the context of a plan. This means that plans can make use of annotations for very interesting and useful things in multi-agent programs. In fact, this allows us to define more specific plans, which require particular annotations: omitting annotation effectively means that the plan is so general as to apply to any annotation. For example, suppose we want to write a plan for an agent that always accepts invitations to go down the pub, provided the invitation is from a good friend; we could have a plan such as:

```
+down_the_pub(Pub) [source(Agent)]
  : good_friend(Agent)
  <- !location(Pub).
```

More typically, we will need to check the annotations we have introduced in the belief base in the plan context. For example, if we have annotated the beliefs with degrees of certainty (say, with a term `degOfCert`), we could have a plan such as:

```
+!contract(Agent)
  : trustworthy(Agent) [degOfCert(C)] & C>0.9
  <- ...
```

If our agent is entering a contract with another agent, it might need different courses of action depending on how certain it already is that it can trust that other agent.

Higher order variables

In *Jason*, as in Prolog, we can bind variables to a predicate (or here, more generally, a formula) rather than a term. This allows us to do rather sophisticated things. For example, we could store an action formula as a term in the belief base, and use a variable to retrieve the action and execute it. Or we can use a variable to match *any* event, as in the example below – even though care should be taken as otherwise the agent might behave in unexpected ways as all events would be matched to that plan.

```
+!G[source(baddy)] : true <- true.
```

The plan tells the agent to ignore any goals that agent ‘baddy’ tries to delegate (in Chapter 6 we explain how agents delegate goals to one another). The keyword `true` used above can be used to denote an empty context, and also an empty body. Alternatively, we can omit those parts of a plan if they are empty. Thus the plan above could have been written simply ‘`+!G[source(baddy)] .`’ or we could have plans such as:

```
+!G[source(S)] : S \== goody.
```

omitting the body – the plan says that all goal delegations from agents other than ‘goody’ should be ignored; or

```
+B <- !~B.
```

omitting the context – this is a plan for a rebellious agent that will try to oppose everything it perceives in the environment. An example using variables bound to actions would be:

```
+B : simple_action_for(B,A) <- A.
```

In this case, obviously the programmer has to ensure that whatever terms `A` might be instantiated with is a valid agent action.

Plan labels

One thing we did not mention when explaining the plan syntax is that we can give a specific label to a plan; we can think of this as a ‘name’ for that plan. In fact, all plans have labels automatically generated by *Jason*, if we do not give specific names for each plan. This is important because, as we shall see later, it is sometimes necessary to be able to refer to a particular plan (instance) that the agent is running. Being able to specify labels is important not only because it might help us if we use meaningful names for the plans, but because we can also associate meta-level information about the plans within plan labels. The general notation for plans with a specific label is as follows:

```
@label te : ctxt <- body.
```

However, the label itself does not have to be a simple term such as ‘`label`’: it can have the same form of a predicate. That is, it can include annotations too, and it is in the annotations that we should include meta-level information (e.g. information that can help the interpreter choose one applicable plan over another), as exemplified below:

```

@shopping(1) [chance_of_success(0.7), usual_payoff(0.9),
              source(ag1), expires(autumn)]
+need(Something)
  :   can_afford(Something)
  <- !buy(Something).

```

and later (in Section 7.2) we see how we can write Java code for this meta-level information about the plan to be used.

Even though we can use a complex predicate, as above, as a label, it is normal practice to use a predicate of arity 0 (i.e. a proposition) but with annotations. For example, `@shopping1[...]` where `shopping1` would effectively be the name of the plan and all relevant meta-level information would be included in the annotations. There are pre-defined plan annotations which have special meaning for the interpreter; we discuss these in Chapter 4.4.

We have now seen all of the language constructs that can be used to program individual agents in *Jason*. The next section shows a complete agent program, which will clarify the role of these constructs and how they all come together.

3.4 Example: A Complete Agent Program

Consider the following example (inspired by [29]), which comes with *Jason*'s distribution.

A domestic robot has the goal of serving beer to its owner. Its mission is quite simple, it just receives some beer requests from the owner, goes to the fridge, takes out a bottle of beer, and brings it back to the owner. However, the robot should also be concerned with the beer stock (and eventually order more beer using the supermarket's home delivery service) and some rules hard-wired into the robot by the Department of Health (in this example this rule defines the limit of daily beer consumption).

We use the Prometheus notation [75] in Figure 3.3 to give more details of the scenario. The system is composed of three agents: the robot, the owner and the supermarket. The possible perceptions that the agents have are:

- `at(robot, Place)`: to simplify the example, only two places are perceived, `fridge` (when the robot is in front of the fridge) and `owner` (when the robot is next to the owner). Thus, depending on its location in the house, the robot

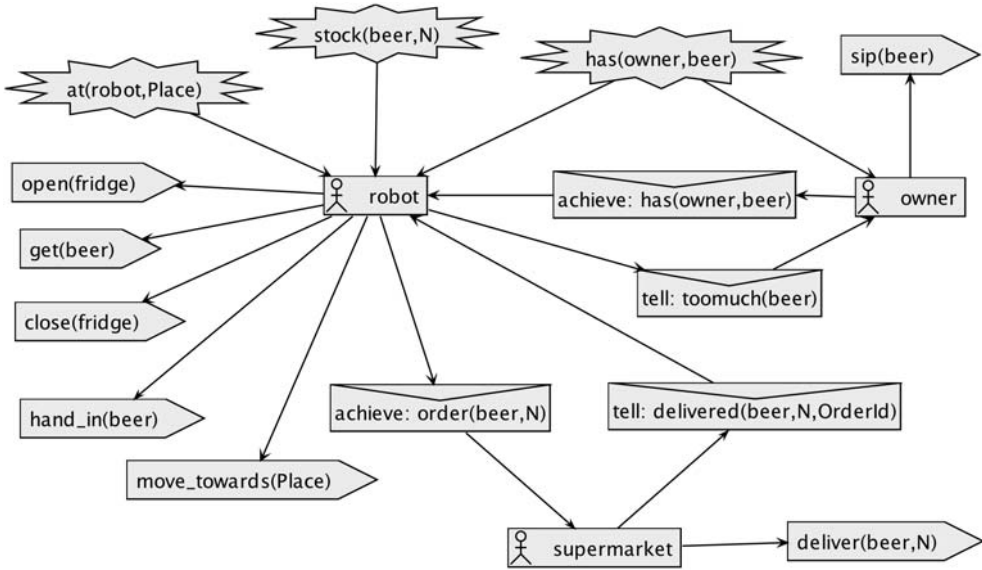


Figure 3.3 Overview of the domestic robot application.

will perceive either `at(robot, fridge)` or `at(robot, owner)`, or of course no at percept at all (in case it is in neither of those places);

- `stock(beer, N)`: when the fridge is open, the robot will perceive how many beers are stored in the fridge (the quantity is represented by the variable `N`);
- `has(owner, beer)`: is perceived by the robot and the owner when the owner has a (non-empty) bottle of beer.

Most of the actions have obvious meanings, except perhaps `move_towards`. Again, to simplify the example, the way the robot moves in the house is greatly simplified. We assume that, when it performs the action `move_towards(fridge)`, its hardware (or the simulator) performs one step towards the fridge, so the robot does not need to be concerned with the path to reach the fridge.

Besides perception and action, the agents also send messages to each other (messages are denoted by ‘envelopes’ in Figure 3.3). The owner sends a message to the robot when he wants another beer. The message type is ‘achieve’, so when the robot receives this message it will have `has(owner, beer)` as a new goal. A similar message is sent to the supermarket agent when the robot wants to order more beer. The supermarket sends a ‘tell’ message to robot when it has delivered the order. Tell messages are used to change the agent’s beliefs (`delivered(beer, N, OrderId)` in this case). The AgentSpeak code for the agents in this application is as follows.

Owner Agent

```

!get(beer). // initial goal

/* Plans */

@g
+!get(beer) : true
  <- .send(robot, achieve, has(owner,beer)).

@h1
+has(owner,beer) : true
  <- !drink(beer).
@h2
-has(owner,beer) : true
  <- !get(beer).

// while I have beer, sip
@d1
+!drink(beer) : has(owner,beer)
  <- sip(beer);
    !drink(beer).
@d2
+!drink(beer) : not has(owner,beer)
  <- true.

+msg(M)[source(Ag)] : true
  <- .print("Message from ",Ag," : ",M);
    -msg(M).

```

The owner's initial goal is `get(beer)`. Thus when it starts running the event `+!get(beer)` is generated and the plan labelled `@g` is triggered (most plans in the source are identified by a label prefixed by '@'). This plan simply sends a message to the robot asking it to achieve the goal `has(owner,beer)`.

When the robot has achieved the goal, the owner perceives this, and the belief `has(owner,beer)[source(percept)]` is added to the belief base. This change in the belief base generates an event that triggers plan `@h1`. When executing `@h1`, the subgoal `!drink(beer)` is created, and then the owner starts drinking, by recursively executing plan `@d1`. The event `+!drink(beer)` has two relevant plans (`@d1` and `@d2`) and they are selected based on their context: `@d1` is selected whenever the owner has any beer left and `@d2` otherwise.

The beer will eventually be finished (the percept `has(owner, beer)` is removed), and the intention of drinking also finishes (the `!drink(beer)` plan recursion stops). The event `-has(owner, beer)` then triggers plan `@h2`. This plan creates a new subgoal (`!get(beer)`), which starts the owner behaviour from the beginning again.

The last plan simply prints a special type of message received from an agent identified in the source as `Ag`. Note that the message is removed from the belief base after the `.print` internal action is executed. This is necessary because, when such message arrives, an event is generated only in the case where that message is not already in the belief base.

Supermarket Agent

```
last_order_id(1). // initial belief

// plan to achieve the the goal "order" from agent Ag
+!order(Product,Qtd)[source(Ag)] : true
  <- ?last_order_id(N);
      OrderId = N + 1;
      -+last_order_id(OrderId);
      deliver(Product,Qtd);
      .send(Ag, tell, delivered(Product,Qtd,OrderId)).
```

The code of the supermarket agent is quite simple. It has an initial belief representing the last order number and a plan to achieve the goal of processing an order for some agent. The triggering event of this plan has three variables: the product name, the quantity and the source of the achievement goal (the sender of the achieve message in this case). When selected for execution, the plan consults the belief base for the last order number (`?last_order_id(N)`); increments this number (`OrderId = N + 1`); updates the belief base with this new last order number (`-+last_order_id(OrderId)`); delivers the product (through an *external* action); and finally sends a message to the client (the robot, in this case) telling it that the product has been delivered.

Robot Agent

```
/* Initial beliefs */

// initially, I believe that there are some beers in the fridge
available(beer,fridge).

// my owner should not consume more than 10 beers a day :-)
```

```
limit(beer,10).
```

```
/* Rules */
```

```
too_much(B) :-
```

```
    .date(YY,MM,DD) &
    .count(consumed(YY,MM,DD,_,_,_,B),QtdB) &
    limit(B,Limit) &
    QtdB > Limit.
```

```
/* Plans */
```

```
@h1
```

```
+!has(owner,beer)
    : available(beer,fridge) & not too_much(beer)
    <- !at(robot,fridge);
        open(fridge);
        get(beer);
        close(fridge);
        !at(robot,owner);
        hand_in(beer);
        // remember that another beer will be consumed
        .date(YY,MM,DD); .time(HH,NN,SS);
        +consumed(YY,MM,DD,HH,NN,SS,beer).
```

```
@h2
```

```
+!has(owner,beer)
    : not available(beer,fridge)
    <- .send(supermarket, achieve, order(beer,5));
        !at(robot,fridge). // go to fridge and wait there.
```

```
@h3
```

```
+!has(owner,beer)
    : too_much(beer) & limit(beer,L)
    <- .concat("The Department of Health does not allow me ",
        "to give you more than ", L,
        " beers a day! I am very sorry about that!",M);
        .send(owner,tell,msg(M)).
```

```
@m1
```

```
+!at(robot,P) : at(robot,P) <- true.
```



```

@m2
+!at(robot,P) : not at(robot,P)
  <- move_towards(P);
  !at(robot,P).

// when the supermarket finishes the order, try the 'has'
// goal again
@a1
+delivered(beer,Qtd,OrderId)[source(supermarket)] : true
  <- +available(beer,fridge);
  !has(owner,beer).

// when the fridge is opened, the beer stock is perceived
// and thus the available belief is updated
@a2
+stock(beer,0)
  : available(beer,fridge)
  <- -available(beer,fridge).

@a3
+stock(beer,N)
  : N > 0 & not available(beer,fridge)
  <- +available(beer,fridge).

```

The robot is the main agent in this example, and its code is thus more complex. The robot maintains a belief regarding availability of beer in the fridge: initially, it believes `available(beer,fridge)`. It is important for the robot to remember whether or not there is beer available irrespective of its location in the environment. The robot does not perceive the beer everywhere: the stock is only perceived when the robot is in front of the fridge and it is open. The `stock` perception is removed from the belief base when the robot, for instance, closes the fridge. Thus, it needs to remember what it saw inside the fridge. Plans @a2 and @a3 update the availability belief according to the stock perception.

The belief base also has a rule to determine whether the owner already drank the maximum number of beers allowed per day. This verification is based on the number of `consumed` beliefs. When the robot gives a beer to the owner it adds a belief to remember that; this belief has the form `+consumed(Year,Month,Day,Hour,Minute,Second,beer)` (see the end of plan @h1). The rule gets the current date using the internal action `.date`; then uses another internal action `(.count)` to count how many `consumed` beliefs there are in the belief base (the variable `QtdB`

unifies with this quantity). The `too_much` belief holds if `QtdB` is less than the limit of beers (10 in this example).

The main task of the robot is to achieve the goal `!has(owner,beer)`, created by an `achieve` message received from the owner. The robot has three plans to achieve this goal (`@h1`, `@h2`, and `@h3`). The first plan is selected when there is beer available and the owner has not drunk too much beer today (see the plan context). In this plan execution, the robot goes to the fridge (subgoal), opens it (external action), gets a beer (external action), closes the fridge (external action), goes back to the owner (subgoal) and finally gives the beer to the owner (external action). The plans to achieve the `!at` subgoal are quite simple and use the `move_towards` external action until the agent believes it is in the target place `P`.

The second plan is selected when there is no more beer in the fridge. In this case, the robot asks the supermarket for five more bottles of beer, goes to the fridge, and waits for the beer delivery. This plan finishes when the robot reaches the fridge. However the `!has(owner,beer)` goal should be resumed when the beers are delivered. Plan `@a1` does that when a message from the supermarket is received.

The third plan is (unfortunately) selected when the robot is forbidden to give more beers to the owner. In this case, it says a special message to the owner explaining the reasons.

Running the System

Every system developed with *Jason* has a multi-agent system definition file where the set of participating agents, the shared environment and various parameters for the multi-agent system execution are defined. The various configuration options that can be used in the multi-agent system definition file will be presented throughout the book, and Appendix A.2 has a complete list of such options; for now it is enough to define the three agents that compose the system and the Java class that simulates the environment:¹

```
MAS domestic_robot {

    environment: HouseEnv(gui)

    agents: robot;
           owner;
           supermarket agentArchClass SupermarketArch;
}
```

¹The class that implements the environment is `houseEnv` and is explained in Example 5.2.

An excerpt of the result of the system execution is:

```
[robot] doing: move_towards(fridge)
[robot] doing: move_towards(fridge)
[robot] doing: move_towards(fridge)
[robot] doing: open(fridge)
[robot] doing: get(beer)
[robot] doing: close(fridge)
[robot] doing: move_towards(owner)
[robot] doing: move_towards(owner)
...
[robot] doing: hand_in(beer)
[owner] doing: sip(beer)
[owner] doing: sip(beer)
...
[supermarket] doing: deliver(beer,5) ...
[owner] saying: Message from robot: The Department of
Health does not allow me to give you more than 10 beers
a day! I am very sorry about that!
```

When running the examples available with the *Jason* distribution, and also when developing your own systems, note that multi-agent systems definition files have an extension `.mas2j` (for historical reasons) and files with AgentSpeak code have an extension `.asl`.

3.5 Exercises

Basic

1. The robot used in Example 3.4 has plans (identified by `h1-h3`) triggered when the agent has a goal that unifies `has(owner,beer)` despite the source of the goal (i.e. personal or delegated). Change these plans to be triggered only if the source of the goal is the owner agent.

Basic

2. Improve the code of the Supermarket agent of the example so that it manages its stock. It initially has, for instance, 100 bottles of beer and this value will decrease as it delivers beer to the robot. Of course, it can only deliver beer when it has enough beer in its stock, otherwise it should inform the robot that it has no more beer in its stock.

Basic

3. The robot asks the supermarket for more beer only when the fridge is empty (see `+stock` plans). Modify the robot code so that its behaviour is more proactive and it asks for more beer before it is finished. Note that: (i) it is not a case of simply changing 0 to 1 in plan `@a2`, because even with only 1 beer remaining in the fridge, there is still beer available, so the belief about beer being available should not be removed; (ii) plan `@h2` may be removed from

the code because the robot is now pro-active and there will always be beer available; (iii) when some beer is delivered (see plan @a1), the robot does not need to achieve the goal `has(owner, beer)` anymore.

4. In the robot code, we have two plans for the goal `!at`. What happens if we change their order (`m2` before `m1`)? What happens if the second plan does not have a context, as in the following code? Basic

```
@m1 +!at(robot,P) : at(robot,P) <- true.
@m2 +!at(robot,P) : true
      <- move_towards(P);
      !at(robot,P).
```

Is the behaviour of the robot the same as with the original code? In this case, what happens if we change the order of the plans?

5. Create a new supermarket agent, initially with the same code as the supermarket of the example. Change the code of both supermarkets such that, once they have started, they send the following message to the robot, informing of their price for beer. Advanced

```
.send(robot,tell,price(beer,3))
```

The robot should then buy beer from the cheapest supermarket. To help the robot coding, write a (Prolog-like) rule that finds the cheapest supermarket according to the robot's belief base. Note that the robot will always have two beliefs of price from different sources (the two supermarkets).